RIS Lab II
Report for Assignment 2

Yiheng Cao, Eduard-Antonio Zippenfenig, Wai Huen Sin, Amanuel Basaznew Legesse

***Abstract*— In this assignment, we explored both from GUI and code to control the motion of robots, and find out the difference in Turtlesim simulation and real robots.**

Task 1 Teleoperation in Traffic

Task 1.1 Teleoperation

We launched rviz after installing the required dependency python3-pynput and cloning the duckietown gui package via git. It provides a virtual joystick to control the Duckiebot as shown in Figure 1.

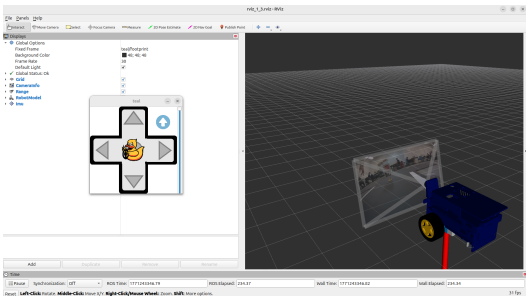


Fig. 1. RViz interface and virtual joystick control during teleoperation

We remotely drove the robot with the joystick. The front camera displays the robot's environment, and provides a real-time visual feedback for navigation without requiring direct line-of-sight.

Task 1.2 Reflection

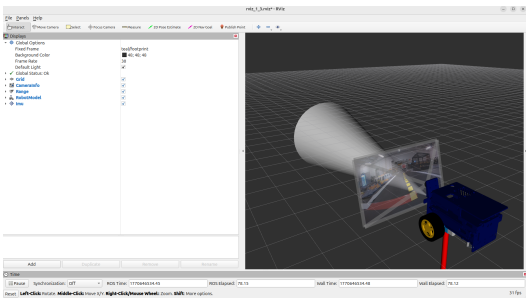


Fig. 2. Rviz interface showing the Duckiebot's environment

Problems for autonomous driving:
Robots' behavior:

1) Traffic Rule Understanding
   - Traffic signs
   - Right-handed traffic laws
   - Speed limits (Initial Speed cannot be so fast)

   Implementation: camera-based detection with encode traffic rules to the robots.
2) Obstacles avoidance:
   - Detect static or dynamic obstacles
   - Interact safely with other moving robots

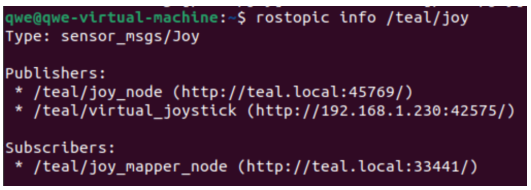
   Implementation: range sensing and object detection to adjust speed/direction
3) Road intersection handling:
   - Precise turning (left/right/straight/U-turn)

   Implementation: using a finite state machine to execute turns
4) Emergency handling:
   - Safety mechanisms for unexpected situations

   Implementation: Implement a stop-and-wait behavior

Task 1.3 Introspection

The messages from the user input are transmitted through the following pipeline to eventually move the wheels:



```
qwe@qwe-virtual-machine:~$ rostopic info /teal/joy
Type: sensor_msgs/Joy

Publishers:
 * /teal/joy_node (http://teal.local:45769/)
 * /teal/virtual_joystick (http://192.168.1.230:42575/)

Subscribers:
 * /teal/joy_mapper_node (http://teal.local:33441/)
```

Fig. 3. rostopic info /teal/joy

User input: The /teal/virtual_joystick and /teal/joy_node publishes raw input to the /teal/joy topic.

Mapping: The /teal/joy_mapper_node subscribes to

```
qwe@qwe-virtual-machine:~$ rostopic info /teal/joy_mapper_node/car_cmd
Type: duckietown_msgs/Twist2DStamped

Publishers:
 * /teal/joy_mapper_node (http://teal.local:33441/)

Subscribers:
 * /teal/car_cmd_switch_node (http://teal.local:41849/)
```

Fig. 4. rostopic info /teal/joy_mapper_node/car_cmd

this topic, trnaslates the data, and publishes it to /teal/joy_mapper_node/car_cmd.

Switching: The /teal/car_cmd_switch_node receives these commands and routes the active control signal to /teal/car_cmd_switch_node/cmd.

Kinematics: The /teal/kinematics_node processes the switch command and publishes the calculated left/right wheel speeds to /teal/wheels_driver_node/wheels_cmd.

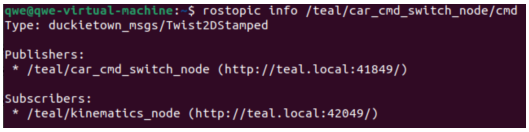


Fig. 5. rostopic info /teal/car_cmd_switch_node/cmd

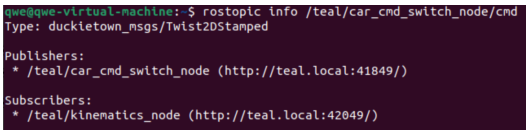


Fig. 6. rostopic info /teal/kinematics_node/velocity

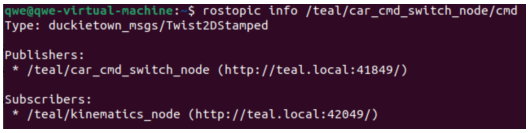


Fig. 7. rostopic info /teal/wheels_driver_node/wheels_cmd

Execution: Finally, the /teal/wheels_driver_node subscribes to these commands and physically moves the wheels.

Task 2 Calibration

We inspected the parameters with rosparam get /teal/kinematics_node and observed that the movement of the robot was not consisting of the movement of the right and left wheel, but of tangential velocity (v) and angular velocity (omega) for both wheels. Also, we had the parameter trim which was necessary for this task.

Fig. 8. The parameter of /teal/kinematics_node

Task 2.1 Software Preparation

From the previous task, the following changes were made in the drive_forward.cpp: we included the duckietown_msgs/Twist2DStamped as a header file, we changed the type of variables to Twist2DStamped, we used the cmd.v and cmd.omega for the velocities mentioned above. Omega was set to 0 since the robot was programmed to move forward.

Task 2.2 Calibration

After saving the change in the .cpp file, and building the workspace, we moved to calibrating the trim value. Through trial and error, we defined the tangential velocity with the value 0.3, taking into account the friction with the mats, and the time of movement of 8 seconds.

We chose values in the interval [-1,1]. We observed for the value 0 that the distance from the robot position to the smaller, and as we went up from 0 to 1 it grew again, therefore we tested for values circling 0 to get to the best one, but we chose 0 as the ideal value.

Trim values: [-1, -0.75, -0.5, -0.25, 0, 0.25, 0.01, 0.001, 0.0001]

Distance from the axis: [0, 0, 0, 60, 5, 36, 29, 40, 68] (measured in cm)

For the first 3 values, our robot moved in circles, hence we didn't take them as optimal values.

We measured another 5 times for trim=0 and values changed significantly, showing that the calibration cannot be to precise.

Distance from the axis: [5, 10, 8, 40, 35, -23, 10] (measured in cm)

Task 2.3 + 2.4 Reporting, Evaluation and Stability

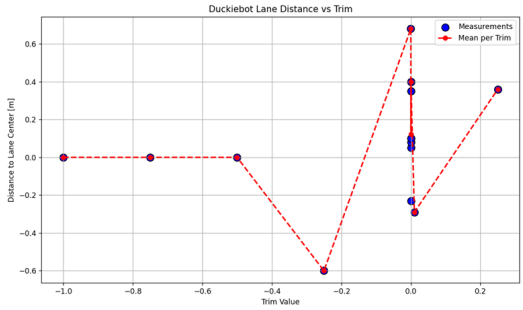


Fig. 9. The relation between Trim value and Distance to Lane Center(cm)

After having done the trial measurements with the range of trim values, the final decision for the trim value was set to 0. Due to the variable nature seen from the standard deviation, the value of 0 wouldn't consistently work as a viable value.

From the measurements obtained from the trim value of 0, there are: 0.05, 0.10, 0.08, 0.40, 0.35, -0.23, 0.10

Calculating the mean value gives: 0.1214.

Once we have that the standard deviation can be computed as:

$$\sigma = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \overline{x})^2}{n-1}}$$

With this, a final value of 0.21 m is obtained.

Task 3 Revisiting Old Tasks

One of the main differences between the turtlesim from Lab I and the duckiebot is the kinematics used with the turtlesim using linear.x/y/z and the duckiebot with a tangential velocity (V) and angular speed (omega). This combined with the additional fact that the turtlesim was step based instead of time based, gave the final output a large contrast. The angles that the duckiebot had to move resulted in it over as well as undershooting the required turns. Finally, network reliability plays a critical role. Unlike local simulation, the Duckiebot suffers from latency and packet loss. The rostopic hz also shows the communication frenquency of that is inconsistent. Besides that, the real-world physical rules such

as friction and battery power can both introduce Duckiebot moving instability.

CONTRIBUTIONS

- Yiheng Cao - completion for 1.3 and 3
- Eduard-Antonio Zippenfenig - completion for 2.1 and 2.2
- Wai Huen Sin - completion for 1.1 and 1.2
- Amanuel Basaznew Legesse - completion for 2.3, 2.4 and 3